

Technical Assignment

Document Splitting & Metadata Generation Pipeline

Context

You are provided with a set of PDF documents (one file per document) and a single Excel extraction file (**output_test.xlsx**) produced by a classification/OCR pipeline. Each row of the Excel represents an annotation referring to a file (**platform filename**) and, in most cases, to a specific page (**page**). Rows are distinguished by **label container type**: document classification (*document_classification*), page classification (*page_classification*) and entity extraction (*entity_extraction*).

Objective

Split each PDF into its logical sub-documents based on the classification information contained in the Excel file, and produce, for each generated sub-document, an Excel file holding only the metadata that pertains to that subset of pages.

Requirement 1 — PDF splitting

Splitting must be driven by page classification (*page_classification*) together with the invoice number extracted for each page. An invoice is typically identified by a **Fattura** page that carries an invoice number (*entity_extraction* → label container **Fattura**, attribute **Numero fattura**): that page opens an invoice sub-document, and the following contiguous **Fattura** pages are absorbed into the same invoice. When a **Fattura** page bearing a **new** invoice number is encountered, a new invoice has started. Pages classified as **Altro** are **not** included in the invoice sub-documents: they are handled separately according to the configurable **Altro** mode (see Requirement 5). As a result, a single input PDF may produce one or more invoice PDFs.

There may (or may not) be edge cases where a single invoice is interrupted by non-Fattura documents (e.g. **Altro** pages appearing in the middle of the same invoice number). How to handle such cases is left to the candidate's discretion, provided the chosen behaviour is documented.

Fidelity constraint: sub-documents must preserve the exact original content of the pages — fonts, colours, any images and the position of the page number. Pages must be extracted from the original PDF, not rebuilt or regenerated.

Requirement 2 — One Excel per split

For each generated sub-document, produce an Excel file containing all and only the rows of the source Excel related to that subset of pages (match on **platform filename** + **page** belonging to the split range), keeping the original columns. You must define and document how *document_classification* rows (which have an empty **page** and refer to the whole file) are handled.

Requirement 3 — Configurable Python script

The task must be implemented as a Python script that runs the entire flow automatically. The script must not contain parameters or paths hard-coded in the source: all configuration must be read from an external **.ini** file (e.g. **config.ini**), which must contain at least:

- path of the input Excel file;
- path of the ZIP archive containing the PDFs to be split;
- output directory for the split PDFs;
- output directory for the split Excel files;
- path/name of the final output ZIP;
- the parameter controlling the handling of **Altro** pages (see below).

The script must handle reading of the **.ini** file robustly (sections, missing values, non-existent paths) with clear error messages.

Requirement 4 — Input

The process input consists of two items: the Excel extraction file and the ZIP archive with the PDFs to be split. The script must extract the PDFs from the ZIP and match them to the Excel rows through **platform filename**.

Requirement 5 — Handling of *Altro* pages (configurable)

The treatment of pages classified as **Altro** must be selectable from the **.ini** file through three modes:

- 1 **discard** — **Altro** pages are completely excluded from the outputs (neither in the split PDFs nor in the related Excel files);
- 2 **split** — **Altro** pages are kept and split into their own separate sub-documents (following the same run-based logic), each with its corresponding Excel; they are never merged into the invoice sub-documents;
- 3 **group** — for each source PDF, a single additional PDF is generated containing only the **Altro** pages of that file (with the corresponding Excel holding only the metadata of those pages).

The selected mode must be reflected consistently in both the PDFs and the Excel files produced.

Requirement 6 — Output

The output must be produced in two forms simultaneously: the loose files on the filesystem in the directories indicated in the configuration (the split PDFs and the related Excel files), and a single ZIP file collecting all the output. Use a coherent, reconstructable naming convention pairing each PDF with its Excel, for example **{filename}_part{N}.pdf** and **{filename}_part{N}.xlsx**.

Requirement 7 — Run instructions

The deliverable must include clear instructions to launch the project: how to install the dependencies (e.g. a **requirements.txt**), how to configure the **.ini** file, and the exact command(s) to run the script end to end (for example):

```
pip install -r requirements.txt
python split_documents.py --config config.ini
```

A short README describing prerequisites, configuration keys and expected inputs/outputs is expected.

Edge cases to handle (and document)

- PDFs may contain more pages than the Excel describes: unclassified pages must be identified and handled (decide whether to exclude, attach or isolate them, with a rationale).
- Files that do not begin with a **Fattura** page.
- Consistency between the actual page count of the PDF and the maximum **page** value in the Excel: provide a check.

Evaluation criteria

- Correctness of the splitting logic with respect to classification.
- Fidelity of extracted pages (fonts, colours, images and page numbers intact).
- Correctness and completeness of metadata in the individual Excel files.
- Proper externalisation of configuration (no hard-coded paths) and robust reading of the .ini file and ZIP input.
- Correct and consistent implementation of the three **Altro** handling modes.
- Production of both the loose files and the final ZIP.
- Presence of clear run instructions; code cleanliness and reproducibility.